

FurkanSaglam2526630

November 16, 2025

1 Assignment 2: N-grams and Language Identification

1.1 CNG463 - Introduction to Natural Language Processing

1.1.1 METU NCC Computer Engineering | Fall 2025-26

Student Name: Furkan Sağlam **Student ID:** 2526630

Due Date: 16 November 2025 (Sunday) before midnight

1.2 Overview

This assignment focuses on: 1. Building **character-based** 2-gram and 3-gram language models with Laplace smoothing 2. Sentence-based language identification using 10-fold cross-validation 3. Evaluation using accuracy, precision, recall, and F1-score 4. Comparison and analysis

Note: For language identification, we use **character n-grams** rather than word n-grams because they better capture language-specific patterns like letter combinations, diacritics, and writing systems.

Grading: - Written Questions (7×4 pts): **28 pts** - Code Tasks with TODO (11 total): **72 pts** distributed by effort level: - Simple tasks: 4 pts each (2 cells) - Moderate tasks: 6 pts each (4 cells) - Complex tasks: 8 pts each (5 cells) - **Total: 100 pts**

1.3 Pre-Submission Checklist

- ☐ Name and student ID at top
- ☐ No cells are added or removed
- ☐ All TODO sections completed
- ☐ All questions answered
- ☐ Code runs without errors
- ☐ Results tables included
- ☐ Run All before saving

1.4 Setup and Imports

```
[ ]: # Standard libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from collections import defaultdict, Counter
from typing import List, Tuple, Dict
import re

# Scikit-learn for cross-validation and metrics
from sklearn.model_selection import KFold
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
from scipy.stats import ttest_rel

# Set random seed for reproducibility
np.random.seed(42)
```

2 Task 1: Corpus Preparation and Statistics (22 points)

2.1 1.1: Upload Corpus Files

Prepare your text files in **two different languages** (accepted formats: .txt, .pdf, or .docx). When you run the cell below, you'll be prompted to upload files for each language separately. Make sure your files contain substantial text (reports, essays, or similar content from other courses). Each language requires at least **5000** words in its corpus.

```
[1]: from google.colab import files

print("Upload your ENGLISH corpus file(s):")
english_files = files.upload()

print("\nUpload your SECOND LANGUAGE corpus file(s):")
second_lang_files = files.upload()
```

Upload your ENGLISH corpus file(s):

<IPython.core.display.HTML object>

Saving A_Survey_of_Video_Game_Testing.pdf to A_Survey_of_Video_Game_Testing.pdf

Upload your SECOND LANGUAGE corpus file(s):

<IPython.core.display.HTML object>

Saving 10.18603-sanatvetasarim.1215230-2815435.pdf to
10.18603-sanatvetasarim.1215230-2815435.pdf

2.2 1.2: Load and Preprocess Data (12 points)

Load your uploaded files, extract text, preprocess, split into sentences, and tokenize. You'll need helper functions to handle different file formats.

Steps: 1. Read files based on format (.txt, .pdf, .docx) and combine them into single text for each language 2. Apply preprocessing (e.g., lowercasing, handling punctuation) 3. Split each corpus into individual sentences 4. Tokenize each sentence into words (for statistics) 5. Store the results as two lists of tokenized sentences

Important: You'll use word tokenization for calculating statistics, but for the n-gram models in Task 2, you'll work with character n-grams directly on the sentence strings.

```
[71]: import re
from typing import List
from PyPDF2 import PdfReader
from docx import Document
def read_txt_file(filename: str) -> str:
    """Read a .txt file and return its content."""
    with open(filename, 'r', encoding='utf-8') as f:
        return f.read()

def read_pdf_file(filename: str) -> str:
    """Read a .pdf file and return its text content."""
    # TODO: Install and use PyPDF2 or pdfplumber
    # Example: pip install PyPDF2
    file = open(filename, "rb")

    r= PdfReader(file)

    t = ''
    for i in range(len(r.pages)):
        page = r.pages[i]
        text = page.extract_text()
        t+=text.lower()
    return t

def read_docx_file(filename: str) -> str:
    """Read a .docx file and return its text content."""
    # TODO: Install and use python-docx
    # Example: pip install python-docx
    r = Document(filename)
    t = ''
    for i in range(len(r.paragraphs)):
```

```

        text = r.paragraphs[i].text.lower()
        t += text
    return t

def split_into_sentences(text: str) -> List[str]:
    """Split text into sentences."""
    # TODO: Implement sentence splitting
    # You can use simple regex or nltk.sent_tokenize
    pattern = r"[.!?]"
    result = re.split(pattern, text)
    sentences = []
    for sentence in result:
        if sentence != '':
            sentences.append(sentence)
    return sentences
text = "Ali geldi. Veli gitti! Sen gördün mü?"

print(split_into_sentences(text))

def tokenize_sentence(sentence: str) -> List[str]:
    """Tokenize a sentence into words."""
    # TODO: Implement word tokenization
    # You can use str.split() or nltk.word_tokenize
    return sentence.split()
text = "Ali geldi. Veli gitti. Sen gördün mü?"
print(tokenize_sentence(text))

# TODO:
#
# 1. Read and combine files for each language
#    - Loop through lang1_files and lang2_files
#    - Use appropriate read function based on file extension
#    - Combine all text into lang1_text and lang2_text
#
# 2. Apply preprocessing to both lang1_text and lang2_text
#    (e.g., lowercasing, removing extra whitespace)
#
# 3. Split each corpus into sentences using split_into_sentences()
#
# 4. Tokenize each sentence using tokenize_sentence()
#
# Note: These tokenized sentences will be used for statistics in Task 1.
# In Task 2, you'll work with the raw sentence strings for character n-grams.
#
# At the end, you should have:

```

```
# lang1_sentences = [sent1, sent2, ...]
# lang2_sentences = [sent1, sent2, ...]
# lang1_sentences_tokenized = [[word1, word2, ...], [word1, word2, ...], ...]
# lang2_sentences_tokenized = [[word1, word2, ...], [word1, word2, ...], ...]

# [8 pts]
```

```
['Ali geldi', ' Veli gitti', ' Sen gördün mü']
['Ali', 'geldi.', 'Veli', 'gitti.', 'Sen', 'gördün', 'mü?']
```

Question 1.1: What preprocessing choices did you make and why? (3-5 sentences)

I made everything lowercase. Therefore, if there are the same characters, they will match. For example, normally Man and man are different, but after lowercasing they become the same word. Since their meaning is the same, this is important for getting correct results.

2.3 1.3: Basic Statistics (10 points)

Calculate and display key statistics for both language corpora to understand their characteristics.

```
[73]: # TODO: Calculate statistics for BOTH languages
#
# For each language (lang1_sentences and lang2_sentences):
# - Total character count
# - Special character/punctuation count
# - Character vocabulary size (unique characters)
# - Total word count
# - Word vocabulary size (unique words)
# - Sentence count
# - Average sentence length (in words)
#
# Example structure:
# lang1_total_characters = sum(len(sentence) for sentence in lang1_sentences)
# ...
# lang1_total_words = sum(len(sentence) for sentence in
    ↪ lang1_sentences_tokenized)
# lang1_word_vocabulary = len(set(word for sentence in
    ↪ lang1_sentences_tokenized for word in sentence))
# ...
#
# Print statistics side by side for the two corpora
Englishtextpdf = read_pdf_file("A_Survey_of_Video_Game_Testing.pdf")
Turkishtextpdf = read_pdf_file("10.18603-sanatvetasarim.1215230-2815435.pdf")
lang1_sentences = split_into_sentences(Englishtextpdf)
lang2_sentences = split_into_sentences(Turkishtextpdf)
text = ["Ali aşşl dkşl asd", "asdlaş skldjlas."]
text2 = "Ali gol atı. Demir üzgün. Eda çalışıyor."
text_sentences = split_into_sentences(text2)
```

```

text_sentences_tokenized = [tokenize_sentence(sentence) for sentence in
    ↪text_sentences]
clnlang1_sentences = []
clnlang2_sentences = []
for sentence in lang1_sentences:
    cleaned = sentence.replace(" ", "")
    clnlang1_sentences.append(cleaned)
for sentence in lang2_sentences:
    cleaned = sentence.replace(" ", "")
    clnlang2_sentences.append(cleaned)
lang2_total_characters = sum(len(sentence) for sentence in clnlang2_sentences)
lang1_total_characters = sum(len(sentence) for sentence in clnlang1_sentences)
result = []
for sentence in text:
    pattern = r"[0-9A-Za-zÀ-ÖØ-öø-ÿİıĞğŞşÇçÖöÜü ]"

    result.append(re.sub(pattern, "", sentence))
print(result)
print(sum(len(sentence) for sentence in result) )
print(f"Language 1 Total Characters: {lang1_total_characters}")
print(f"Language 2 Total Characters: {lang2_total_characters}")

# 3

alltext = ""
cleanspacedata = []
for sentence in text:
    cleaned = sentence.replace(" ", "")
    cleanspacedata.append(cleaned)
for sentence in cleanspacedata:
    alltext+=sentence

#Total word count
lang1_sentences_tokenized = [tokenize_sentence(sentence) for sentence in
    ↪lang1_sentences]
lang1_total_words = sum(len(sentence) for sentence in lang1_sentences_tokenized)
print("Language 1 word count :",lang1_total_words)

lang2_sentences_tokenized = [tokenize_sentence(sentence) for sentence in
    ↪lang2_sentences]
lang2_total_words = sum(len(sentence) for sentence in lang2_sentences_tokenized)
print("Language 2 word count :",lang2_total_words)
# Word vocabulary size (unique words)
lang1_word_vocabulary = len(set(word for sentence in lang1_sentences_tokenized,
    ↪for word in sentence))

```

```

lang2_word_vocabulary = len(set(word for sentence in lang2_sentences_tokenized_
    ↪for word in sentence))
print("Language 1 unique words :", lang1_word_vocabulary)
print("Language 2 unique words :", lang2_word_vocabulary)

#Sentence count
lang1_sentence_count = len(lang1_sentences)
lang2_sentence_count = len(lang2_sentences)
print("Language 1's sentence count", lang1_sentence_count)
print("Language 2's sentence count", lang2_sentence_count)

# Average sentence length (in words)
avragesentence = lang1_total_words / lang1_sentence_count
print("Language 1's average sentence length", avragesentence)
avragesentence2 = lang2_total_words / lang2_sentence_count
print("Language 2's average sentence length", avragesentence2)
# [6 pts]

```

```

['', '.']
1
Language 1 Total Characters: 49828
Language 2 Total Characters: 41220
Language 1 word count : 8953
Language 2 word count : 6015
Language 1 unique words : 2847
Language 2 unique words : 2787
Language 1's sentence count 822
Language 2's sentence count 476
Language 1's average sentence length 10.891727493917275
Language 2's average sentence length 12.63655462184874

```

Question 1.2: What are the key differences between your two corpora? (2-3 sentences)

The English PDF has more words and sentences than the Turkish PDF, but the Turkish PDF has a more average sentence length than English Pdf. Even the English PDF has more total words, both languages have a small different word number of unique words.

3 Task 2: Character N-gram Language Identification (58 points)

Baseline (46 pts): Implement character-based 2-gram and 3-gram models, run 10-fold CV, report accuracy.

Creativity (12 pts): Out-of-vocabulary analysis.

3.1 2.1: Implement Character N-gram Models (12 points)

Implement the `CharNgramLanguageModel` class with Laplace smoothing using NLTK's n-gram utilities. The model should count **character** n-grams during training and calculate sentence probabil-

ities with smoothing.

Key difference from word n-grams: Instead of tokenizing sentences into words, you'll work with individual characters in each sentence.

```
[7]: import nltk
from nltk.util import ngrams, pad_sequence
from nltk.lm import Laplace
from nltk.lm.preprocessing import padded_everygram_pipeline
from typing import List

from math import log, exp

# Download required NLTK data
nltk.download('punkt', quiet=True)

class CharNgramLanguageModel:
    """
    Character-based N-gram language model with Laplace (add-1) smoothing using
    ↪NLTK.
    """
    text = ["Ali gol attı", "Veli sevindi"]
    t = []

    def __init__(self, n: int = 2):
        """
        Initialize the character n-gram model.

        Args:
            n: Order of n-gram (2 for bigram, 3 for trigram)
        """
        self.n = n
        self.model = Laplace(n)

    def train(self, sentences: List[str]):
        """
        Train the model on a list of sentences.

        Args:
            sentences: List of sentences (each sentence is a string)
        """
        # TODO: Your code here
        # Convert each sentence to a list of characters
        # Example: "hello" -> ['h', 'e', 'l', 'l', 'o']
        # Use padded_everygram_pipeline to prepare training data with padding
        # Then fit the model using self.model.fit()
```

```

# Convert each sentence to a list of characters
c = []
for sentence in sentences:
    c.append(list(sentence))
train, vocab = padded_everygram_pipeline(self.n, c)
self.model.fit(train, vocab)

def get_probability(self, sentence: str) -> float:
    """
    Calculate the probability of a sentence.

    Args:
        sentence: Sentence string

    Returns:
        Probability of the sentence
    """
    # TODO: Your code here
    # Convert sentence to list of characters
    # Pad the character sequence and generate n-grams
    # For each n-gram, get probability using self.model.score()
    # Multiply probabilities together (or sum log probabilities to avoid
    ↪underflow)
    c = list(sentence)
    size = self.n - 1
    paddedchars = (["<s>"] * size) + c + (["</s>"] * size)
    setofngrams = list(ngrams(paddedchars, self.n))
    plog = 0.0
    for ng in setofngrams:
        history = ng[:-1]
        ch = ng[-1]
        p = self.model.score(ch, history)

        if p > 0:
            plog += log(p)
        else:
            return 0.0

    prob = exp(plog)
    return prob

```

```
# [8 pts]
```

3.1.1 Spot Check: Inspect Your N-gram Models

After implementing the model, train sample models on both languages and inspect what they learned.

```
[8]: import random
# TODO: Train sample models and inspect them
#
# 1. Create 2-gram and 3-gram models for both languages
# 2. Train them on your full datasets (lang1_sentences and lang2_sentences)
# 3. Inspect the models to see what n-grams they learned
#
# Example:
# model_2gram_lang1 = NgramLanguageModel(n=2)
# model_2gram_lang1.train(lang1_sentences)
# model_3gram_lang1 = NgramLanguageModel(n=3)
# model_3gram_lang1.train(lang1_sentences)
#
# model_2gram_lang2 = NgramLanguageModel(n=2)
# model_2gram_lang2.train(lang2_sentences)
# model_3gram_lang2 = NgramLanguageModel(n=3)
# model_3gram_lang2.train(lang3_sentences)
#
# Display sample n-grams and their counts from each model
# Check vocabulary size: len(model.model.vocab)
# Show most common n-grams or test probabilities on sample sentences

# Your code here

model_2gram_lang1 = CharNgramLanguageModel(n=2)
model_2gram_lang1.train(lang1_sentences)
model_3gram_lang1 = CharNgramLanguageModel(n=3)
model_3gram_lang1.train(lang1_sentences)
model_2gram_lang2 = CharNgramLanguageModel(n=2)
model_2gram_lang2.train(lang2_sentences)
model_3gram_lang2 = CharNgramLanguageModel(n=3)
model_3gram_lang2.train(lang2_sentences)

sizelang1n2 = len(model_2gram_lang1.model.vocab)
sizelang1n3 = len(model_3gram_lang1.model.vocab)

sizelang2n2 = len(model_2gram_lang2.model.vocab)
sizelang2n3 = len(model_3gram_lang2.model.vocab)
```

```

print("size of Language 1 for n=2 ",sizelang1n2)
print("size of Language 1 for n=3 ",sizelang1n3)

print("size of Language 2 for n=2 ",sizelang2n2)
print("size of Language 2 for n=3 ",sizelang2n3)
testsentenceoflang1 = random.choice(lang1_sentences)
testsentenceoflang2 = random.choice(lang2_sentences)

print("Test sentence1", testsentenceoflang1)

print("Test sentence2",testsentenceoflang2)

print("n2gramlang1:", model_2gram_lang1.get_probability(testsentenceoflang1))
print("n2gramlang2:", model_2gram_lang2.get_probability(testsentenceoflang2))

print("n3gramlang1:", model_3gram_lang1.get_probability(testsentenceoflang1))
print("n3gramlang2:", model_3gram_lang2.get_probability(testsentenceoflang2))

text = ["Ali gol attı","Arzu sevindi"]
model_2gram_test = CharNgramLanguageModel(n=2)
model_2gram_test.train(text)
model_3gram_test = CharNgramLanguageModel(n=3)
model_3gram_test.train(text)

testsentenceoftest = random.choice(text)
print("-----")
print("Test :",testsentenceoftest)
print("P_2gram_test:", model_2gram_test.get_probability(testsentenceoftest))
print("P_3gram_test:", model_3gram_test.get_probability(testsentenceoftest))

# [4 pts]

```

```

size of Language 1 for n=2  96
size of Language 1 for n=3  96
size of Language 2 for n=2  97
size of Language 2 for n=3  97
Test sentence1  3) Automation is overlooked; game testing currently relies
mostly on human testers (Section V-C);
4) The search for the "fun-factor" and "game balance" is

```

mainly executed by game-play testers (Section V-D);
 5) There is no one-size-ts-all testing process for game projects as the game types differ greatly (Section V-E);
 6) Game studios acknowledge the importance of testing, opportunity for open-source initiatives (Section V-F);
 7) Issues like lack of plan and poor testing coverage call for game testing to be performed early (Section V-G);
 We conclude that automating the video-game testing process is the natural next step

Test sentence2

Macklin, C

n2gramlang1: 0.0

n2gramlang2: 5.089940694163539e-16

n3gramlang1: 0.0

n3gramlang2: 4.7217531468754555e-18

Test : Ali gol att1

P_2gram_test: 4.580199877150394e-14

P_3gram_test: 6.902943200548197e-15

3.2 2.2: Implement Language Identification (8 points)

Create a function that compares sentence probabilities from two language models and returns the predicted label.

```
[62]: def identify_language(sentence: str,
                             model_lang1: CharNgramLanguageModel,
                             model_lang2: CharNgramLanguageModel) -> int:
    """
    Identify the language of a sentence using two character-based language
    ↪models.

    Args:
        sentence: Sentence string
        model_lang1: Language model for language 1 (label 0)
        model_lang2: Language model for language 2 (label 1)

    Returns:
        Predicted label (0 or 1)
    """
    # TODO: Your code here
    #
    # Steps:
    # 1. Calculate probability of sentence using model_lang1
    #     prob1 = model_lang1.get_probability(sentence)
    #
    # 2. Calculate probability of sentence using model_lang2
    #     prob2 = model_lang2.get_probability(sentence)
```

```

#
# 3. Compare probabilities and return the label of the model with higher
↪probability
#     if prob1 > prob2: return 0
#     else: return 1

prob1 = model_lang1.get_probability(sentence)

prob2 = model_lang2.get_probability(sentence)
if prob1 > prob2:
    return 0
else:
    return 1

# [8 pts]

```

3.3 2.3: Implement Evaluation Function (6 points)

Create a function that calculates accuracy, precision, recall, and F1-score given predicted and true labels.

```

[10]: from typing import List, Dict
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

def calculate_metrics(y_true: List[int], y_pred: List[int]) -> Dict[str, float]:
    """
    Calculate evaluation metrics.

    Args:
        y_true: True labels
        y_pred: Predicted labels

    Returns:
        Dictionary with accuracy, precision, recall, f1_score
    """
    # TODO: Your code here
    # Use sklearn's accuracy_score and precision_recall_fscore_support
    accuracy = accuracy_score(y_true, y_pred)
    precision, recall, f1_score, _ = precision_recall_fscore_support(y_true,
↪y_pred, average='weighted', zero_division=0)
    return {'accuracy': accuracy, 'precision': precision, 'recall': recall,
↪'f1_score': f1_score}

```

```
# [6 pts]
```

3.4 2.4: 10-Fold Cross-Validation for Language Identification (8 points)

Implement 10-fold cross-validation to evaluate your character-based n-gram models. In each fold, split the data, train separate models for each language and n-gram order, make predictions, and evaluate performance.

```
[11]: from sklearn.model_selection import KFold

# Prepare dataset: combine sentence STRINGS from both languages with labels
X = lang1_sentences + lang2_sentences
y = [0] * len(lang1_sentences) + [1] * len(lang2_sentences)

print(f"Dataset prepared:")
print(f"  Total sentences: {len(X)}")
print(f"  Language 1 (label 0): {sum(1 for label in y if label == 0)}_
    ↪ sentences")
print(f"  Language 2 (label 1): {sum(1 for label in y if label == 1)}_
    ↪ sentences")
print()

# Initialize 10-fold cross-validation
kfold = KFold(n_splits=10, shuffle=True, random_state=42)

# Store results for each fold
results_2gram = {'accuracy': [], 'precision': [], 'recall': [], 'f1': []}
results_3gram = {'accuracy': [], 'precision': [], 'recall': [], 'f1': []}

# TODO: Implement 10-fold cross-validation
#
# For each fold, you need to:
# 1. Split data into train and test sets using train_idx and test_idx from_
    ↪ kfold.split(X)
# 2. Separate training sentences by language based on labels
# 3. Train FOUR models total:
#    - Two 2-gram models (one per language)
#    - Two 3-gram models (one per language)
# 4. Make predictions on test sentences using identify_language()
# 5. Calculate metrics using calculate_metrics()
# 6. Store results in results_2gram and results_3gram dictionaries
#

for fold_idx, (train_idx, test_idx) in enumerate(kfold.split(X), 1):
    print(f"\n{'='*50}")
    print(f"Fold {fold_idx}/10")
    print(f"{'='*50}")
```

```

# TODO Your implementation here
#1
xtrain = []
ytrain = []
ytest= []
xtest=[]
for i in train_idx:
    xtrain.append(X[i])
    ytrain.append(y[i])
for i in test_idx:
    xtest.append(X[i])
    ytest.append(y[i])
#2
trainEnglish = []
trainTurkish = []
for i in range(len(xtrain)):
    if ytrain[i] == 0:
        trainEnglish.append(xtrain[i])
    else:
        trainTurkish.append(xtrain[i])
#3
Englishn2 = CharNgramLanguageModel(n=2)
Englishn2.train(trainEnglish)

Turkishn2 = CharNgramLanguageModel(n=2)
Turkishn2.train(trainTurkish)

Englishn3 = CharNgramLanguageModel(n=3)
Englishn3.train(trainEnglish)

Turkishn3 = CharNgramLanguageModel(n=3)
Turkishn3.train(trainTurkish)

#4
ypn2 = []
ypn3 = []
for sentence in xtest:
    p2 = identify_language(sentence,Englishn2,Turkishn2)
    ypn2.append(p2)

    p3 = identify_language(sentence,Englishn3,Turkishn3)
    ypn3.append(p3)
#5

```

```

metricsoffn2 = calculate_metrics(ytest,ypn2)
metricsoffn3 = calculate_metrics(ytest,ypn3)
#6
print("Result for 2-N")
print("acc", metricsoffn2['accuracy'])
print("prec", metricsoffn2['precision'])
print("recall", metricsoffn2['recall'])
print("f1", metricsoffn2['f1_score'])

print("Result for 3-N")
print("acc", metricsoffn3['accuracy'])
print("prec", metricsoffn3['precision'])
print("recall", metricsoffn3['recall'])
print("f1", metricsoffn3['f1_score'])

results_2gram['accuracy'].append(metricsoffn2['accuracy'])
results_2gram['precision'].append(metricsoffn2['precision'])
results_2gram['recall'].append(metricsoffn2['recall'])
results_2gram['f1'].append(metricsoffn2['f1_score'])
results_3gram['accuracy'].append(metricsoffn3['accuracy'])
results_3gram['precision'].append(metricsoffn3['precision'])
results_3gram['recall'].append(metricsoffn3['recall'])
results_3gram['f1'].append(metricsoffn3['f1_score'])

print("\n" + "="*50)
print("Cross-validation completed!")
print("="*50)

# [8 pts]

```

Dataset prepared:

Total sentences: 1298

Language 1 (label 0): 822 sentences

Language 2 (label 1): 476 sentences

```

=====
Fold 1/10
=====

Result for 2-N
acc 0.8307692307692308
prec 0.8277379116088794
recall 0.8307692307692308
f1 0.8272435897435897
Result for 3-N

```

acc 0.8769230769230769
prec 0.8769230769230769
recall 0.8769230769230769
f1 0.8769230769230769

=====
Fold 2/10
=====

Result for 2-N
acc 0.8153846153846154
prec 0.8153846153846154
recall 0.8153846153846154
f1 0.8153846153846154
Result for 3-N
acc 0.8846153846153846
prec 0.8844072594072593
recall 0.8846153846153846
f1 0.8830611533154407

=====
Fold 3/10
=====

Result for 2-N
acc 0.8538461538461538
prec 0.8531203092550326
recall 0.8538461538461538
f1 0.8534285991730679
Result for 3-N
acc 0.8923076923076924
prec 0.8915084915084914
recall 0.8923076923076924
f1 0.8916743774803118

=====
Fold 4/10
=====

Result for 2-N
acc 0.8
prec 0.8008203024865419
recall 0.8
f1 0.7972201538843386
Result for 3-N
acc 0.8461538461538461
prec 0.8486778538047444
recall 0.8461538461538461
f1 0.8440155029879528

=====

Fold 5/10

Result for 2-N

acc 0.8307692307692308
prec 0.8278296074634747
recall 0.8307692307692308
f1 0.8284188034188034
Result for 3-N
acc 0.9153846153846154
prec 0.9155746317036639
recall 0.9153846153846154
f1 0.9138756341218053

Fold 6/10

Result for 2-N

acc 0.8153846153846154
prec 0.8153846153846154
recall 0.8153846153846154
f1 0.8153846153846154
Result for 3-N
acc 0.8692307692307693
prec 0.8687269960726256
recall 0.8692307692307693
f1 0.8675726640223682

Fold 7/10

Result for 2-N

acc 0.8769230769230769
prec 0.8761663890773754
recall 0.8769230769230769
f1 0.8756359979889392
Result for 3-N
acc 0.9230769230769231
prec 0.9234059177924455
recall 0.9230769230769231
f1 0.922272498743087

Fold 8/10

Result for 2-N

acc 0.8307692307692308
prec 0.829126687266222
recall 0.8307692307692308

```
f1 0.8291049530179965
Result for 3-N
acc 0.8769230769230769
prec 0.8763050902585787
recall 0.8769230769230769
f1 0.8757126931039975
```

```
=====
Fold 9/10
=====
```

```
Result for 2-N
acc 0.8449612403100775
prec 0.8458748615725359
recall 0.8449612403100775
f1 0.8435092039743202
Result for 3-N
acc 0.8527131782945736
prec 0.8524547803617571
recall 0.8527131782945736
f1 0.85253180175162
```

```
=====
Fold 10/10
=====
```

```
Result for 2-N
acc 0.875968992248062
prec 0.8752478817378764
recall 0.875968992248062
f1 0.8746893072474468
Result for 3-N
acc 0.8914728682170543
prec 0.8908822443706165
recall 0.8914728682170543
f1 0.8909470033825899
```

```
=====
Cross-validation completed!
=====
```

3.5 2.5: Display Results (12)

Create a table showing for each model: Mean accuracy, precision, recall, F1 (with std)

```
[12]: import pandas as pd
      # TODO: Calculate and display summary statistics
      #
      #
      # Example:
```

```

# results_df = pd.DataFrame({
#     'Model': ['2-gram', '3-gram'],
#     'Accuracy': [...],
#     'Precision': [...],
#     ...
# })
avgac2 = sum(results_2gram['accuracy']) / len(results_2gram['accuracy'])
avgp2 = sum(results_2gram['precision']) / len(results_2gram['precision'])
avgr2 = sum(results_2gram['recall']) / len(results_2gram['recall'])
avgf12 = sum(results_2gram['f1']) / len(results_2gram['f1'])

avgac3 = sum(results_3gram['accuracy']) / len(results_3gram['accuracy'])
avgp3 = sum(results_3gram['precision']) / len(results_3gram['precision'])
avgr3 = sum(results_3gram['recall']) / len(results_3gram['recall'])
avgf13 = sum(results_3gram['f1']) / len(results_3gram['f1'])
results_df = pd.DataFrame({
    'Model': ['2-gram', '3-gram'],
    'Accuracy': [avgac2, avgp3],
    'Precision': [avgp2, avgp3],
    'Recall': [avgr2, avgr3],
    'F1-score': [avgf12, avgf13]
})

print(results_df)

# [4 pts]

```

	Model	Accuracy	Precision	Recall	F1-score
0	2-gram	0.837478	0.836669	0.837478	0.836002
1	3-gram	0.882887	0.882887	0.882880	0.881859

Question 2.1: Which of your trained models performed best on the validation data, and why? (3-4 sentences)

The 3-gram model performed best. It works with three characters, which gives more information than the 2-gram model. The 2-gram model can miss some language patterns.

Question 2.2: Were the results consistent across different folds of cross-validation? (2-3 sentences)

Yes, the results were the results consistent across different folds of cross-validation. Both models have small differences in validation for floods

3.6 2.6: Out-of-Vocabulary Testing (12 pts)

Test your models with **five** sentences containing characters or character combinations not common in your training corpus. For character n-grams, this might include unusual letter combinations, foreign words, or made-up words that still follow language patterns.

```
[13]: # TODO: Create and test OOV sentences
#
# - Be creative!

ovvsentences = ["la pasi3n keeps relationsip alive","Fenecbace ŝampiyon_
↳olamkısı uk az kaldı","The man said ###!!!!,,,, them","Ma son anda Cycle_
↳Deck sayesinde kazandı","İch hasse zwie brother"]

for sentence in ovvsentences:
    print()
    print("-----")

    print(sentence)
    print("Englis probability n=2 ",model_2gram_lang1.get_probability(sentence))
    print("Turkish probability n=2 ",model_2gram_lang2.
↳get_probability(sentence))
    print("English probability n=3 ",model_3gram_lang1.
↳get_probability(sentence))
    print("Turkish probability n=3",model_3gram_lang2.get_probability(sentence))

    print("Perdict for n=2 if it is English,0 or İf it Turkish, it is 1. Result_
↳",identify_language(sentence, model_2gram_lang1, model_2gram_lang2))
    print("Perdict for n=2 if it is English,0 or İf it Turkish, it is 1. Result_
↳",identify_language(sentence, model_3gram_lang1, model_3gram_lang2))
    print("-----")
    print()

# [8 pts]
```

```
-----
la pasi3n keeps relationsip alive
Englis probability n=2  1.510146502200327e-46
Turkish probability n=2  4.545655074816743e-50
English probability n=3  3.0768074570189054e-50
Turkish probability n=3  9.341877278287108e-59
Perdict for n=2 if it is English,0 or İf it Turkish, it is 1. Result  0
Perdict for n=2 if it is English,0 or İf it Turkish, it is 1. Result  0
-----
```

```
-----
Fenecbace ŝampiyon olamkısı uk az kaldı
Englis probability n=2  2.2114344235147535e-78
Turkish probability n=2  3.343071591186063e-60
```

```

English probability n=3 1.350812177881813e-82
Turkish probability n=3 2.26354907377145e-68
Perdict for n=2 if it is English,0 or İf it Turkish, it is 1. Result 1
Perdict for n=2 if it is English,0 or İf it Turkish, it is 1. Result 1
-----

```

```

-----
The man said ###!!!,,,, them
Englis probability n=2 2.1771293337537882e-48
Turkish probability n=2 1.5307989752226079e-50
English probability n=3 1.8020371581364243e-48
Turkish probability n=3 2.145917593486744e-53
Perdict for n=2 if it is English,0 or İf it Turkish, it is 1. Result 0
Perdict for n=2 if it is English,0 or İf it Turkish, it is 1. Result 0
-----

```

```

-----
Maç son anda Cycle Deck sayesinde kazandı
Englis probability n=2 7.325942596523632e-65
Turkish probability n=2 2.308198734583583e-54
English probability n=3 9.379309848724544e-69
Turkish probability n=3 8.385903522645373e-61
Perdict for n=2 if it is English,0 or İf it Turkish, it is 1. Result 1
Perdict for n=2 if it is English,0 or İf it Turkish, it is 1. Result 1
-----

```

```

-----
İch hasse zwie brother
Englis probability n=2 3.2480306379899083e-34
Turkish probability n=2 9.673744349816538e-35
English probability n=3 2.379430521684272e-38
Turkish probability n=3 8.331938072052696e-41
Perdict for n=2 if it is English,0 or İf it Turkish, it is 1. Result 0
Perdict for n=2 if it is English,0 or İf it Turkish, it is 1. Result 0
-----

```

Question 2.3: How well did your models handle out-of-vocabulary (OOV)

samples? (2-3 sentences)

In here, we tried 5 sentences. Even if the same sentence has different language or words, both our model chooses the correct prediction. Even they have small differences between English and Turkish, they can detect the correct language

4 Task 3: Statistical Analysis (20 points)

Baseline (10 pts): Statistical significance testing and comparison.

Creativity (10 pts): Advanced analysis (confusion matrices, error analysis, etc.).

4.1 3.1: Statistical Significance Testing (10 points)

Use paired t-test to compare models. $p\text{-value} < 0.05$ indicates statistically significant difference.

```
[14]: from scipy.stats import ttest_rel

# TODO: Perform paired t-tests
#
# Compare: 2-gram vs 3-gram
#
# Use: t_stat, p_value = ttest_rel(results_1, results_2)
t_accuracy, p_accuracy = ttest_rel(results_2gram['accuracy'],
    ↪results_3gram['accuracy'])
print("Accuracy: t ", t_accuracy, " p = ", p_accuracy)
t_precision, p_precision = ttest_rel(results_2gram['precision'],
    ↪results_3gram['precision'])
print("Precision: t = ", t_precision, " p = ", p_precision)

t_recall, p_recall = ttest_rel(results_2gram['recall'], results_3gram['recall'])
print("Recall: t =", t_recall, " p =", p_recall)

t_f1, p_f1 = ttest_rel(results_2gram['f1'], results_3gram['f1'])
print("F1-score: t =", t_f1, " p =", p_f1)

# [6 pts]
```

```
Accuracy: t  -6.3934792338809565  p =  0.0001262368597118747
Precision: t = -6.278664593642292  p =  0.00014460670292058695
Recall: t = -6.3934792338809565    p = 0.0001262368597118747
F1-score: t = -6.5645056795267855  p = 0.00010342227518522975
```

Question 3.1: Are the performance differences statistically significant? Explain what ‘statistical significance’ means in this context. (2-3 sentences)

Yes, the output shows a statistically significant difference. All p value smaller than 0.05, so it is not a random creation. This output shows that 3 grams is better than 2 gram.

4.2 3.2: Advanced Analysis (10 points)

Perform deeper analysis such as per-language performance, misclassification patterns, etc.

```

[75]: # TODO: Your advanced analysis here

testtext = ["Temu", "Hepsiburada", "Amazon sells books.", "Aile", "Aile modu_
↳az", "Designer player deneyimini improve etmek için yeni mechanic ekledi.
↳", "Test sürecindeki bug dowladı ekip için.", "Developerlar gameplay akışını_
↳analytics sonuçlarına göre tekrar tasarladı.", "The doctor recovery sürecimin_
↳surprisingly fast olduğunu söyledi."]
print(testtext[0])

print(identify_language(testtext[0], model_2gram_lang1, model_2gram_lang2))
print(identify_language(testtext[0], model_3gram_lang1, model_3gram_lang2))

print(testtext[1])

print(identify_language(testtext[1], model_2gram_lang1, model_2gram_lang2))
print(identify_language(testtext[1], model_3gram_lang1, model_3gram_lang2))

print(testtext[2])

print(identify_language(testtext[2], model_2gram_lang1, model_2gram_lang2))
print(identify_language(testtext[2], model_3gram_lang1, model_3gram_lang2))

print(testtext[3])

print(identify_language(testtext[3], model_2gram_lang1, model_2gram_lang2))
print(identify_language(testtext[3], model_3gram_lang1, model_3gram_lang2))

print(testtext[4])

print(identify_language(testtext[4], model_2gram_lang1, model_2gram_lang2))
print(identify_language(testtext[4], model_3gram_lang1, model_3gram_lang2))

print(testtext[5])

print(identify_language(testtext[5], model_2gram_lang1, model_2gram_lang2))
print(identify_language(testtext[5], model_3gram_lang1, model_3gram_lang2))

# For short words like "Hepsiburada" and "Temu", the 3-gram model predicted_
↳wrong, but 2-gram correct,
# In mixed TR-EN sentences, both models mostly predicted correctly, but in_
↳third is wrong for 2-gram
# They shows 3-gram better in the long sentence: On the other hand 2-gram can_
↳be better in one word.

# [6 pts]

```

```

Temu
0
1
Hepsiburada
1
0
Amazon sells books.
1
0
Aile
0
0
Aile modu az
1
1
Designer player deneyimini improve etmek için yeni mechanic ekledi.
1
1

```

Question 3.2: What interesting patterns or insights did you discover from your results? (4-5 sentences)

We discovered that the 3-gram model is better than the 2-gram model. The 3-gram model is better than the 2-gram In the Accuracy, Precision, Recall, and F1-score results, the reason because to working with 3 characters. We can also say that the 3-gram model is more stable, because while there were larger differences in the folds for the 2-gram model, the 3-gram model showed smaller variations.

5 Convert Your Colab Notebook to PDF

5.0.1 Step 1: Download Your Notebook

- Go to **File** → **Download** → **Download .ipynb**
- Save the file to your computer

5.0.2 Step 2: Upload to Colab

- Click the **folder icon** on the left sidebar
- Click the **upload button**
- Select your downloaded .ipynb file

5.0.3 Step 3: Run the Code Below

- **Uncomment the cell below** and run the cell
- This will take about 1-2 minutes to install required packages
- When prompted, type your notebook name (e.g.**gs_000000_as2.ipynb**) and press Enter

5.0.4 The PDF will be automatically downloaded to your computer

```
[ ]: # # Install required packages (this takes about 30 seconds)
# print("Installing PDF converter... please wait...")
# !apt-get update -qq
# !apt-get install -y texlive-xetex texlive-fonts-recommended
#   ↪ texlive-plain-generic pandoc > /dev/null 2>&1
# !pip install -q nbconvert

# print("\n" + "="*50)

# # Get notebook name from user
# notebook_name = input("\nEnter your notebook name: ")

# # Add .ipynb if missing
# if not notebook_name.endswith('.ipynb'):
#     notebook_name += '.ipynb'

# import os
# notebook_path = f'/content/{notebook_name}'

# # Check if file exists
# if not os.path.exists(notebook_path):
#     print(f"\n Error: '{notebook_name}' not found in /content/")
#     print("\nMake sure you uploaded the file using the folder icon ( ) on the
#   ↪ left!")
# else:
#     print(f"\n Found {notebook_name}")
#     print("Converting to PDF... this may take 1-2 minutes...\n")

#     # Convert the notebook to PDF
#     !jupyter nbconvert --to pdf "{notebook_path}"

#     # Download the PDF
#     from google.colab import files
#     pdf_name = notebook_name.replace('.ipynb', '.pdf')
#     pdf_path = f'/content/{pdf_name}'

#     if os.path.exists(pdf_path):
#         print(" SUCCESS! Downloading your PDF now...")
#         files.download(pdf_path)
#         print("\n Done! Check your downloads folder.")
#     else:
#         print(" Error: Could not create PDF")
```

```

##Install required packages (this takes about 30 seconds)
print("Installing PDF converter... please wait...")
!apt-get update -qq
!apt-get install -y texlive-xetex texlive-fonts-recommended_
↳texlive-plain-generic pandoc > /dev/null 2>&1
!pip install -q nbconvert

print("\n" + "="*50)

# # Get notebook name from user
notebook_name = input("\nEnter your notebook name: ")

# # Add .ipynb if missing
if not notebook_name.endswith('.ipynb'):
    notebook_name += '.ipynb'

import os
notebook_path = f'/content/{notebook_name}'

# # Check if file exists
if not os.path.exists(notebook_path):
    print(f"\n Error: '{notebook_name}' not found in /content/")
    print("\nMake sure you uploaded the file using the folder icon ( ) on the_
↳left!")
else:
    print(f"\n Found {notebook_name}")
    print("Converting to PDF... this may take 1-2 minutes...\n")

# # Convert the notebook to PDF
!jupyter nbconvert --to pdf "{notebook_path}"

# # Download the PDF
from google.colab import files
pdf_name = notebook_name.replace('.ipynb', '.pdf')
pdf_path = f'/content/{pdf_name}'

if os.path.exists(pdf_path):
    print(" SUCCESS! Downloading your PDF now...")
    files.download(pdf_path)
    print("\n Done! Check your downloads folder.")
else:
    print(" Error: Could not create PDF")

```

Installing PDF converter... please wait...

W: Skipping acquire of configured file 'main/source/Sources' as repository
'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide

```
it (sources.list entry misspelt?)
```